GRASP: Creator principle

GameViewModel.java

```
92      public void createPlayer(Context context, String name, int difficulty, int spriteIndex) {
93          playerColor = -1;
94          switch (spriteIndex) {
95          case 0:
96              playerColor = ContextCompat.getColor(context, R.color.red);
97              break;
98          case 1:
99              playerColor = ContextCompat.getColor(context, R.color.green);
100             break;
101         case 2:
102             playerColor = ContextCompat.getColor(context, R.color.blue);
103             break;
104         default:
105             break;
106         }
107
108         player = new Player(name, difficulty, new PointF( x: 0.5f,  y: 0.5f));
109         player.setRoom(rooms[roomIndex]);
110         joystick.registerObserver(player);
111
112         for (int i = 0; i < 2; i++) {
113             player.registerObserver(rooms[roomIndex].getEnemies()[i]);
114             rooms[roomIndex].getEnemies()[i].setRoom(rooms[roomIndex]);
115         }
116     }
```

As the *GameViewModel* class contains the view classes that come to play on the game screen, the *Player* class is also an instance of this class. Hence, there exists the method *createPlayer* to specifically delegate the work of creating the *Player* object once the main game is started. It also utilizes the face that it has the necessary information of creating a *Player* class, such as *name, difficulty*, and *spriteIndex*, which are passed onto the *GameViewModel* class. This allows the code to have low coupling, which can reduce the chance of potential bugs affecting multiple classes in the future.

SOLID: Open/Closed Principle

```
public class EnemyFactory {
    no usages
    private int enemyColor;
    5 usages   khaihuyenn
    public Enemy createEnemy(String enemyType, PointF position) {
        switch (enemyType) {
        case "Enemy1":
            //enemyColor = ContextCompat.getColor(context, R.color.purple);
            return new Enemy1(position);
        case "Enemy2":
            //enemyColor = ContextCompat.getColor(context, R.color.orange);
            return new Enemy2(position);
        case "Enemy3":
            //enemyColor = ContextCompat.getColor(context, R.color.cyan);
            return new Enemy3(position);
        case "Enemy4":
            //enemyColor = ContextCompat.getColor(context, R.color.pink);
            return new Enemy4(position);
        default:
            return null;
        }
    }
}
```

In the *EnemyFactory* class, the code follows the Open/Closed Principle from the SOLID principles. The Open/Closed Principle states that a class should be open for extension but closed for modification. In this case, the *EnemyFactory* class is open for extension because it can be easily extended to create new types of enemies without modifying the existing code. By adding new classes such as *Enemy1*, *Enemy2*, *Enemy3*, and *Enemy4* that are inherited from the common Enemy base class, the factory can create instances of these new enemy types without changing its existing implementation.

SOLID: Single Responsibility Principle

```
public class Joystick implements Observable {
    2 usages
    private double joystickCenterToTouchDistance;
    2 usages
    private boolean isPressed;
    4 usages
    private float actuatorX;
    4 usages
    private float actuatorY;
    5 usages
    private List<Observer> observers;

    1 usage  Hamza Qadri +1
    public Joystick() { observers = new ArrayList<>(); }
```

```
    public boolean isPressed(float touchPositionX, float touchPositionY, float centerPositionX,
                             float centerPositionY, float outerCircleRadius) {
        joystickCenterToTouchDistance = Math.sqrt(
                Math.pow(centerPositionX - touchPositionX, 2)
                        + Math.pow(centerPositionY - touchPositionY, 2)
        );
        return joystickCenterToTouchDistance < outerCircleRadius;
    }

    1 usage  anikethavula
    public void setIsPressed(boolean isPressed) { this.isPressed = isPressed; }

    1 usage  anikethavula
    public boolean getIsPressed() { return isPressed; }

    1 usage  anikethavula +2
    public void setActuator(float touchPositionX, float touchPositionY, float centerPositionX,
                            float centerPositionY, float outerCircleRadius) {
        float deltaX = touchPositionX - centerPositionX;
        float deltaY = touchPositionY - centerPositionY;
        float deltaDistance = (float) Math.sqrt(Math.pow(deltaX, 2) + Math.pow(deltaY, 2));

        if (deltaDistance < outerCircleRadius) {
            actuatorX = deltaX / outerCircleRadius;
            actuatorY = deltaY / outerCircleRadius;
        } else {
            actuatorX = deltaX / deltaDistance;
            actuatorY = deltaY / deltaDistance;
        }
    }
```

In the *Joystick* class, the code follows the Single Responsibility Principle of SOLID because the entirety of the code within the class works towards fulfilling only one responsibility and only has a single reason to change. In this case specifically, the *Joystick* class only holds code whose sole responsibility is towards handling the state and behavior of a joystick which is part of the user interface. All of the methods within the Joystick class are only in charge of maintaining the state of the joystick, determining if the joystick is pressed, updating the joystick state, and observing state changes, and in terms of the Single Responsibility Principle, all of the code within the class is solely responsible for managing the state and interaction of the joystick in the user interface.

GRASP: High Cohesion Principal

```
1 usage   TippyRegent583
public void startFuse() {
    if (System.currentTimeMillis() - bombStartTime >= BOMB_DURATION) {
        bombStartTime = System.currentTimeMillis();
        state = 1;
        System.out.println("Bomb Started");
    }
}

1 usage   TippyRegent583
public void blowUp(Player p) {
    if (System.currentTimeMillis() - bombStartTime >= EXPLOSION_DURATION){
        bombStartTime = System.currentTimeMillis();
        state = 2;
        if (Math.abs(getPosition().x - p.getPosition().x) < 0.2 && Math.abs(getPosition().y
                - p.getPosition().y) < 0.2) {
            int difficulty = p.getDifficulty();
            int deltaHP = 0;
            //if (!isProtected) {
            switch (difficulty) {
                case 0: // easy
                    deltaHP = -20;
                    break;
                case 1: // medium
```

The Bomb class follows the High Cohesion Principal, as all the methods within the class deal with the action of the bomb blowing up. The class displays temporal cohesion as methods represent different stages of the bomb blowing up. This can be seen in the screenshot above, as the “blowUp” method is the natural follow up to the “startFuse” method in the timeline of a Bomb exploding.

GRASP: Controller principle

```
4 usages  2 implementations  Hamza Qadri
public interface TouchEventHandler {
    2 usages  Hamza Qadri
    default boolean handle(MotionEvent event) {
        switch (event.getAction()) {
        case MotionEvent.ACTION_DOWN:
            onActionDown(event);
            return true;
        case MotionEvent.ACTION_MOVE:
            onActionMove(event);
            return true;
        case MotionEvent.ACTION_UP:
            onActionUp(event);
            return true;
        default:
            return false;
        }
    }

    1 usage  2 overrides  Hamza Qadri
    default void onActionDown(MotionEvent event) {}
    1 usage  1 override  Hamza Qadri
    default void onActionMove(MotionEvent event) {}
    1 usage  1 override  Hamza Qadri
    default void onActionUp(MotionEvent event) {}
}
```

```
    @Override
    public void onActionDown(MotionEvent event) {
        if (viewModel.isJoystickPressed(event.getX(), event.getY(),
                joystickPosition.x, joystickPosition.y, joystickOuterRadius)) {
            viewModel.setJoystickIsPressed(true);
        } else {
            viewModel.useWeapon();
        }
    }

    1 usage  Hamza Qadri
    @Override
    public void onActionMove(MotionEvent event) {
        if (viewModel.getIsJoystickPressed()) {
            viewModel.setJoystickActuator(event.getX(), event.getY(),
                    joystickPosition.x, joystickPosition.y, joystickOuterRadius);
        }
    }

    1 usage  Hamza Qadri
    @Override
    public void onActionUp(MotionEvent event) {
        viewModel.setJoystickIsPressed(false);
        viewModel.resetJoystickActuator();
    }
```

TouchEventHandler is an interface whose implementations receive touch events from views and handle them accordingly. The first image shows this interface, including the default method handle(event) which transfers code execution to one of the other methods. The second shows a snippet of an implementation, GameEventHandler, which handles different types of events received from GameView. Under this implementation, touch events are dispatched to non-UI controller classes, so this is an example of the GRASP Controller pattern.

Decorator Design Pattern for the implementation of Power-ups

PowerUp.java

```java
package com.example.dungeongame.models;

import ...

20 usages  4 implementations  khaihuyenn
public interface PowerUp {
    4 implementations  khaihuyenn
    PointF getPosition();
    6 usages  4 implementations  khaihuyenn
    void applyPowerUp(Player player);
    2 usages  4 implementations  khaihuyenn
    float getCrossSize();
    8 usages  4 implementations  khaihuyenn
    boolean isCollected();
    4 implementations  khaihuyenn
    void update(Observable observable);
}
```

PowerUpDecorator.java

```java
package com.example.dungeongame.models;


import ...

1 usage  khaihuyenn
public class PowerUpDecorator implements PowerUp {
    6 usages
    private PowerUp wrappedPowerUp;

    no usages  khaihuyenn
    public PowerUpDecorator(PowerUp wrappedPowerUp) { this.wrappedPowerUp = wrappedPowerUp; }

    khaihuyenn
    @Override
    public PointF getPosition() { return wrappedPowerUp.getPosition(); }

    6 usages  khaihuyenn
    @Override
    public void applyPowerUp(Player player) { wrappedPowerUp.applyPowerUp(player); }

    2 usages  khaihuyenn
    @Override
    public float getCrossSize() { return wrappedPowerUp.getCrossSize(); }

    8 usages  khaihuyenn
    @Override
    public boolean isCollected() { return wrappedPowerUp.isCollected(); }

    khaihuyenn
    @Override
    public void update(Observable observable) { wrappedPowerUp.update(observable); }

}
```


HealthPowerUp.java
public class HealthPowerUp implements PowerUp {
no usages
private PowerUp powerUp;
3 usages
private boolean collected;
1 usage
private static final float CROSS_SIZE = 100.0f;
1 usage
private static final int HP_INCREASE = 30;
2 usages
private PointF position;
1 usage
private Paint paint;
2 usages khaihuyenn
public HealthPowerUp(PointF position) {...}
khaihuyenn
public PointF getPosition() { return position; }
6 usages khaihuyenn
public void applyPowerUp(Player player) {
player.increaseHP(HP_INCREASE);
collected = true;
}
2 usages khaihuyenn
public float getCrossSize() { return CROSS_SIZE; }
8 usages khaihuyenn
public boolean isCollected() { return collected; }
khaihuyenn
public void update(Observable observable) {
if (observable instanceof Player) {
Player player = (Player) observable;
if (checkCollision(player)) {
applyPowerUp(player);
}
}
}
1 usage khaihuyenn
private boolean checkCollision(Player player) {
float distanceX = Math.abs(getPosition().x - player.getPosition().x);
float distanceY = Math.abs(getPosition().y - player.getPosition().y);
return (distanceX < 0.025 && distanceY < 0.025);
}

FreezeEnemiesPowerUp.java

```java
public class FreezeEnemiesPowerUp implements PowerUp {
    6 usages
    private boolean collected;
    1 usage
    private static final float FROZEN_SIZE = 1.0f;
    2 usages
    private static final long FREEZE_DURATION = 3000; //3 seconds

    2 usages
    private PointF position;
    1 usage
    private Paint paint;
    4 usages
    private long freezeStartTime;

    3 usages  khaihuyenn
    public FreezeEnemiesPowerUp(PointF position) {...}
    khaihuyenn
    public PointF getPosition() { return position; }
    6 usages  khaihuyenn
    public void applyPowerUp(Player player) {
        //Implement logic for freezing enemies
        // Check if the power-up is collected and not already applied
        if (!collected && System.currentTimeMillis() - freezeStartTime >= FREEZE_DURATION) {
            // Apply the freezing logic to enemies
            for (Observer observer : player.getObservers()) {
                if (observer instanceof Enemy) {
                    Enemy enemy = (Enemy) observer;
                    enemy.freeze();
                }
            }

            // Mark the power-up as collected
            collected = true;
            freezeStartTime = System.currentTimeMillis();
        }
    }
    2 usages  khaihuyenn
    public float getCrossSize() { return FROZEN_SIZE; }
    8 usages  khaihuyenn
    public boolean isCollected() { return collected; }

    khaihuyenn
    public void update(Observable observable) {
        if (observable instanceof Player) {
            Player player = (Player) observable;
            if (checkCollision(player)) {
                applyPowerUp(player);
            }
        }
        // Check and handle freezing duration
        if (collected && System.currentTimeMillis() - freezeStartTime >= FREEZE_DURATION) {
            unfreezeEnemies(observable.getObservers());
            collected = false;
        }
    }
    1 usage  khaihuyenn
    private void unfreezeEnemies(Iterable<Observer> observers) {
        for (Observer observer : observers) {
            if (observer instanceof Enemy) {
                ((Enemy) observer).unfreeze();
            }
        }
    }
    1 usage  khaihuyenn
    private boolean checkCollision(Player player) {
        float distanceX = Math.abs(getPosition().x - player.getPosition().x);
        float distanceY = Math.abs(getPosition().y - player.getPosition().y);
        return (distanceX < 0.045 && distanceY < 0.045);
    }
}
```

ShieldPowerUp.java

```java
public class ShieldPowerUp implements PowerUp {
    6 usages
    private boolean collected;
    1 usage
    private static final float SHIELD_SIZE = 1.0f;
    2 usages
    private static final long PROTECTION_DURATION = 3000; // 3 seconds

    2 usages
    private PointF position;
    1 usage
    private Paint paint;
    4 usages
    private long protectionStartTime;
    2 usages  khaihuyenn
    public ShieldPowerUp(PointF position) {
        this.position = position;
        this.paint = new Paint();
        this.collected = false;
        this.protectionStartTime = 0;
    }
    khaihuyenn
    public PointF getPosition() { return position; }
    6 usages  khaihuyenn
    public void applyPowerUp(Player player) {
        //Implement logic for protecting the player from enemies' damage
        if (!collected && System.currentTimeMillis() - protectionStartTime >= PROTECTION_DURATION) {
            for (Observer observer : player.getObservers()) {
                if (observer instanceof Enemy) {
                    Enemy enemy = (Enemy) observer;
                    enemy.protect();
                }
            }
            collected = true;
            protectionStartTime = System.currentTimeMillis();
        }
    }
    2 usages  khaihuyenn
    public float getCrossSize() { return SHIELD_SIZE; }
    8 usages  khaihuyenn
    public boolean isCollected() { return collected; }
```

```java
    khaihuyenn
    public void update(Observable observable) {
        if (observable instanceof Player) {
            Player player = (Player) observable;
            if (checkCollision(player)) {
                applyPowerUp(player);
            }
        }
        if (collected && System.currentTimeMillis() - protectionStartTime >= PROTECTION_DURATION) {
            unProtectPlayer(observable.getObservers());
            collected = false;
        }
    }
    1 usage  khaihuyenn
    private void unProtectPlayer(Iterable<Observer> observers) {
        for (Observer observer : observers) {
            if (observer instanceof Enemy) {
                ((Enemy) observer).unprotect();
            }
        }
    }
    1 usage  khaihuyenn
    private boolean checkCollision(Player player) {
        float distanceX = Math.abs(getPosition().x - player.getPosition().x);
        float distanceY = Math.abs(getPosition().y - player.getPosition().y);
        return (distanceX < 0.045 && distanceY < 0.045);
    }

}
```

In our code, the Decorator design pattern is effectively employed through the *PowerUp* interface, *PowerUpDecorator* class, and specific concrete power-up classes such as *HealthPowerUp*, *FreezeEnemiesPowerUp*, and *ShieldPowerUp*. The *PowerUp* interface defines the common contract for power-ups, ensuring consistency among different power-up types. The *PowerUpDecorator* class acts as a base decorator, implementing the basic functionalities and serving as the foundation for specific decorators. Concrete power-up classes, like *HealthPowerUp, FreezeEnemiesPowerUp,* and *ShieldPowerUp*, extend the *PowerUpDecorator* class and add specific behaviors or attributes to the underlying power-up instances. This allows for a modular and extensible approach to enhancing the functionality of power-ups. For instance, the *ShieldPowerUp* class decorates a power-up with shield functionality, and by combining decorators, the system can seamlessly stack multiple power-ups on a single entity, enabling diverse and dynamic interactions within the game environment.